Protege 온톨로지

연안사고 온톨로지 설계

이론 기초 & 온톨로지 설계

RDF · OWL · Protege 인터페이스 · 클래스 · Property

온톨로지란?

"ChatGPT한테 연안사고 종류를 물어보면 잘 대답합니다. 그러면 온톨로지는 왜 필요할까요? LLM이 다 해결해주는 거 아닌가요?"

① 구조를 모른다

갯바위 추락 vs 체험활동 익수 — 같은 종류?
LLM은 텍스트 유사도로 판단
OWL은 owl:disjointWith로 구조 보장

② 추론을 못 한다

RAG는 있는 것을 찾는다
온톨로지는 없는 것을 추론한다
안전관리 미배치 사업자 자동 발견

③ 이기종 데이터 통합

해경: '익수' / 지자체: '물에 빠짐'
같은 개념, 다른 표현
공통 온톨로지가 다리 역할

④ 학습 데이터 정규화

'방파제 추락' = '방파제에서 떨어짐'
두 표현을 같은 클래스로 매핑
AI 모델 성능 향상에 직접 기여

⑤ 설명 가능 AI (XAI)

딥러닝: '고위험' 출력, 이유 없음
OWL 추론: 갯바위 + 사고 2건 이상
→ HighRiskLocation 분류 근거 제시

RDF 트리플 개념

Subject — Predicate — Object

rdf:	rdfs:	owl:	xsd:
RDF 기본 개념	클래스, 상속	OWL 논리 표현	숫자, 날짜 타입

JSON (의미 없음)

```
{ "사고ID": "ACC001",
  "유형": "익수",
  "장소": "갯벌" }

// "갯벌"은 그냥 문자열.
// 컴퓨터가 의미를 모름
```

RDF Turtle (의미 있음)

```
:ACC001 rdf:type :CoastalAccident .
:ACC001 :occurredAt :TidalFlat .
:TidalFlat rdf:type :CoastalLocation .
:TidalFlat :managedBy :KCG .

// 갯벌 = 연안지형, 해경 관할
```

; 와 , 로 줄여쓰기

```
:ACC001 rdf:type :CoastalAccident ;
        :occurredAt :TidalFlat ;   <- ; 주어 반복 생략
        :hasType :Drowning .
:ACC001 :hasVictim :P_A, :P_B .    <- , 서술어 반복 생략
```

실습 과제 (5분)

문서에서 문장 하나를 골라 트리플로 바꿔보세요.
예: 갯벌은 해경이 관할한다
-> :TidalFlat :managedBy :KCG .

RDFS에서 OWL로 — 왜 필요한가

RDFS로 할 수 있는 것

```
:CoastalAccident rdfs:subClassOf :Accident .
:TidalFlatAccident rdfs:subClassOf
:CoastalAccident .
// 클래스 계층 구조만 표현 가능
```

OWL이 해결하는 것

```
// 필수 조건

SubClassOf: occurredAt some CoastalLocation

// 배타성

:TidalFlatAccident owl:disjointWith
:Underwater.

// 자동 분류 (Defined Class)

MajorAccident EquivalentTo:

  CoastalAccident and (hasCasualty some
integer[>=1])
```

RDFS의 3가지 한계

한계 1

필수 조건 표현 불가

발생장소 없는 사고를 만들어도 오류 없음

한계 2

클래스 배타성 선언 불가

갯벌사고 + 수중사고 동시 성립해도 오류 없음

한계 3

자동 분류 불가

사망자 1명 이상 → 중대사고 자동 분류 불가

핵심 메시지

RDFS = 데이터의 모양을 잡는다
OWL = 데이터의 의미와 규칙을 강제한다
OWL 추론 = LLM hallucination 보완 도구

Protege 설치 & 인터페이스 탐색

주요 탭

Classes
클래스 계층 구조. 오늘 가장 많이 사용

Object Properties
클래스-클래스 관계 정의

Data Properties
클래스-리터럴 값 속성 정의

Individuals
실제 데이터 인스턴스 입력

DL Query
Manchester Syntax 질의 (내일 사용)

설치 & 설정

```
# Java 버전 확인
java -version  (17 이상 필요)

# 새 온톨로지 생성
File -> New
Ontology IRI: http://example.org/coastal

# 저장 형식
File -> Save as -> Turtle (.ttl)
-> @prefix, 트리플이 그대로 저장됨
```

HermiT Reasoner 확인

Reasoner 메뉴 -> HermiT 있으면 정상
없으면: File -> Check for plugins
DL Query 탭: Window -> Tabs -> DL Query

연안사고 문서 분석 & 온톨로지 설계 준비

3가지 시선으로 문서 읽기

명사 찾기

클래스 후보

연안사고, 갯벌, 방파제, 안전관리요원, 해양경찰청

동사/관계 찾기

Object Property 후보

발생하다 → occurredAt, 관할하다 → managedBy

수치/조건 찾기

Data Property + Restriction

5년마다 → hasPlanCycle=5, 1명 이상 → min 1

이름 짓는 관례

```
클래스:            CoastalAccident
                    (첫 글자 대문자, CamelCase)
Object Property:   occurredAt
                    (첫 글자 소문자, camelCase)
Data Property:     hasCasualty   (has 로 시작)
Individual:        Accident_001  (언더바 구분)
```

실습 과제

문서에서 아무 문장 하나를 골라서
클래스 / Property 후보를 각각 뽑아보세요.

옆 사람과 비교해보세요.
같은 문장에서 다르게 뽑을 수 있습니다.
온톨로지 설계에는 정답이 하나가 아닙니다.

클래스 계층 구조 설계 & Protege 입력

설계 3원칙

is-a 관계만

TidalFlatAccident is-a CoastalAccident (O)
CoastalAccident is-a CoastalLocation (X)

3단계 이하

너무 깊은 계층은 관리 어렵고 추론 복잡

같은 기준으로

장소 기준과 유형 기준을 섞으면 안 됨

Protege 입력 팁

부모 클래스 선택 후 [C+] 버튼
클래스 30개 이상이면 Turtle 직접 작성 후 Import
Ctrl+S 자주 저장

연안사고 온톨로지 클래스 계층 (핵심)

```
owl:Thing
├── CoastalAccident (연안사고)
│   ├── LocationBasedAccident
│   │   ├── TidalFlatAccident (갯벌)
│   │   ├── RockyShoreAccident (갯바위)
│   │   └── BreakwaterAccident (방파제)
│   └── ActivityBasedAccident
│       └── CoastalActivityAccident
├── CoastalLocation (연안지형)
│   ├── TidalFlat / RockyShore / Breakwater
├── Person
│   ├── Victim / SafetyManager
└── Organization
    └── KoreaCoastGuard
```

Disjoint Classes 설정

왜 필요한가

```
# Disjoint 없으면 이렇게 해도 오류가 안 납니다
:Accident_001 rdf:type :TidalFlatAccident .
:Accident_001 rdf:type :RockyShoreAccident .  // 갯벌 + 갯바위 동시?
# Disjoint 선언 후에는 추론기가 모순을 잡아냅니다
```

올바른 경우 (형제 클래스)

TidalFlatAccident

owl:disjointWith

RockyShoreAccident (O)

Protege: Ctrl로 다중 선택

-> Make all disjoint

절대 안 되는 경우 (부모-자식)

CoastalAccident

owl:disjointWith

TidalFlatAccident (X)

-> 즉시 모순 발생

-> 온톨로지 전체 Inconsistent

Open World Assumption: 명시하지 않으면 '모른다'. Disjoint도 반드시 명시해야 추론기가 모순을 잡습니다.

Object Property & Data Property 정의

구분	연결 대상	예시
Object Property	클래스 ↔ 클래스	:ACC001 :occurredAt :TidalFlat_001
Data Property	클래스 ↔ 리터럴 값	:ACC001 :hasCasualty "2"^^xsd:integer

Object Property (연안사고 온톨로지)

```
occurredAt
  Domain: CoastalAccident
  Range:  CoastalLocation
hasVictim  Domain: CoastalAccident
           Range:  Victim
managedBy  Domain: CoastalLocation
           Range:  Organization
isLocationOf  inverseOf: occurredAt
```

Data Property (xsd 타입)

```
hasCasualty
  CoastalAccident  xsd:integer
hasAccidentDate
  CoastalAccident  xsd:date
hasWeatherCondition
  CoastalAccident  xsd:string
hasAge    Person  xsd:integer
hasName   Person  xsd:string
```

Domain/Range는 제약이 아닌 추론 힌트

X :occurredAt :TidalFlat_001 (X의 타입 미선언)
-> occurredAt의 Domain이 CoastalAccident이므로
-> 추론기가 X를 자동으로 CoastalAccident로 분류
-> 오류가 아니라 자동 추론이 일어납니다

추론 설계 & Python 연동

Individual · Restriction · Defined Class · rdflib

Individual 생성 및 Property 값 입력

클래스 = 틀, Individual = 실제 사례

```
:Accident_Taean2013 rdf:type :ActivityBasedAccident ;
    :occurredAt :TidalFlat_Taean ;
    :hasVictim :Person_Student001 ;
    :hasCasualty  "5"^^xsd:integer ;
    :hasAccidentDate "2013-07-18"^^xsd:date .

:TidalFlat_Taean rdf:type :TidalFlat ;
    :hasName "태안 갯벌"^^xsd:string .
```

Protege 입력 순서

1. Individuals 탭 -> [+] 클릭
2. 이름 입력 (영문+숫자만)
3. Types + -> 클래스 선택
4. Object property assertions + -> 관계 입력
5. Data property assertions + -> 값 입력 + xsd 타입 지정

자주 하는 실수

이름에 한글 사용 -> IRI 인코딩 오류

Types 미설정 -> 추론기가 분류 못함

Object Property에 숫자값 입력 -> 오류

xsd 타입 미지정 -> 문자열로 저장될 수 있음

OWL Restriction 실습

종류	Manchester Syntax	의미
someValuesFrom	occurredAt some CoastalLocation	반드시 하나 이상 존재해야 함
allValuesFrom	occurredAt only CoastalLocation	있다면 반드시 이 타입이어야 함
exactly	hasCasualty exactly 1	정확히 N개
min / max	hasVictim min 1 Person	최소 / 최대 N개

some (존재 제약)

occurredAt some CoastalLocation

없으면 조건 위반

발생장소 없는 사고 = 연안사고 아님

피자는 토핑을 some 가져야 한다

only (전체 제약)

occurredAt only CoastalLocation

있다면 반드시 이 타입이어야 함

없어도 되지만 있으면 규칙 준수

토핑은 only 식재료여야 한다

입력 방법: Classes 탭 -> 클래스 선택 -> Description -> SubClassOf [+] -> Manchester Syntax 직접 입력 (자동완성 활용)

Defined Class (EquivalentTo)

Primitive vs Defined Class

Primitive Class (SubClassOf)	Defined Class (EquivalentTo)
필요조건만 선언	필요충분조건 선언
단방향: 클래스에 속하면 조건 만족	양방향: 조건 만족하면 자동 분류
조건 만족해도 자동 분류 안 됨	추론기가 자동으로 처리
사람이 직접 타입 지정 필요	데이터 보고 스스로 분류
노란색 아이콘	주황색 아이콘

Defined Class 3가지 예시 (Manchester Syntax)

```
// 중대사고: 사망자 1명 이상
MajorAccident EquivalentTo:
  CoastalAccident and (hasCasualty some xsd:integer[>= 1])
// 미성년자 피해 사고
ChildVictimAccident EquivalentTo:
  CoastalAccident and (hasVictim some Minor)
// 고위험 연안지형: 사고 2건 이상
HighRiskLocation EquivalentTo:
  CoastalLocation and (isLocationOf min 2 CoastalAccident)
```

Intersection & Union 실습

and (Intersection)	or (Union)	not (Complement)
두 조건을 모두 만족 (교집합)	두 조건 중 하나 이상 (합집합)	조건을 만족하지 않는 것 (여집합)

```
// 갯벌에서 발생한 미성년자 피해 사고 (Intersection)
TidalFlatAccident and (hasVictim some Minor)

// 취약계층 피해 사고: 미성년자 또는 노인 (Union)
CoastalAccident and (hasVictim some (Minor or Elder))

// 집중 관리 대상: (사망자>=1 OR 미성년자) AND 관리요원 없음
CoastalAccident
and ( (hasCasualty some xsd:integer[>= 1])
      or (hasVictim some Minor) )
and (hasSafetyManager exactly 0 SafetyManager)
```

Manchester Syntax 팁: 괄호로 우선순위 명확히 | 두세 글자 입력 시 자동완성 | 빨간 밑줄 = 문법 오류

Python rdflib — 온톨로지 로딩 & SPARQL 질의

Step 1 — Turtle 파일 로딩

```
from rdflib import Graph, Namespace, RDF,
RDFS, OWL

g = Graph()
g.parse("coastal.ttl", format="turtle")

print(f"트리플 수: {len(g)}")
# 출력 예: 트리플 수: 187
```

Step 2 — 모든 클래스 조회

```
for cls in sorted(
    g.subjects(RDF.type, OWL.Class)):
  local = str(cls).split('#')[-1]
  label = g.value(cls, RDFS.label)
  print(f'{local} ({label})')

# CoastalAccident (연안사고)
# TidalFlatAccident (갯벌사고)
```

Step 3 — Python으로 트리플 추가

```
from rdflib import Literal, XSD
new = COASTAL.Accident_Python001
g.add((new, RDF.type,
       COASTAL.TidalFlatAccident))
g.add((new, COASTAL.hasCasualty,
       Literal(1, datatype=XSD.integer)))
g.serialize("coastal_updated.ttl",
            format="turtle")
```

이 흐름의 의미

Protege 클릭 = Python 코드 = 같은 파일

FastAPI 연동 -> 온톨로지 기반 API

LangChain 연동 -> GraphRAG

Python rdflib — SPARQL 질의 4가지

질의 1 — 사망자 1명 이상 (FILTER)

```
q = '''SELECT ?acc ?casualty WHERE {
  ?acc :hasCasualty ?casualty .
  FILTER(?casualty >= 1)
} ORDER BY DESC(?casualty)'''
for row in g.query(q):
  print(str(row.acc).split('#')[-1],
        row.casualty)
```

질의 2 — 장소별 사고 건수 (GROUP BY)

```
q = '''SELECT ?loc
  (COUNT(?acc) AS ?cnt)
WHERE { ?acc :occurredAt ?loc . }
GROUP BY ?loc
ORDER BY DESC(?cnt)'''
for row in g.query(q):
  print(str(row.loc).split('#')[-1],row.cnt)
```

질의 3 — 미성년자 피해자 (FILTER)

```
q = '''SELECT ?person ?age
WHERE {
  ?person :hasAge ?age .
  FILTER(?age < 18)
}'''
for row in g.query(q):
  print(row.person, row.age)
```

질의 4 — 종합 현황 (OPTIONAL)

```
q = '''SELECT ?acc ?locName ?casualty
WHERE {
  ?acc rdf:type :CoastalAccident ;
       :occurredAt ?loc .
  OPTIONAL { ?loc :hasName ?locName }
  OPTIONAL { ?acc :hasCasualty ?casualty }
} ORDER BY DESC(?casualty)'''
```

SELECT 조회변수 | WHERE { } 트리플 패턴 | FILTER 조건 | OPTIONAL 선택패턴 | COUNT 집계 | GROUP BY 그룹화 | ORDER BY 정렬

수고하셨습니다

RDF · OWL 개념	트리플, RDFS 한계, OWL 표현력
클래스 설계	계층 구조, Disjoint, Annotation
Property 정의	Object / Data Property, Domain/Range
OWL 논리 표현	Restriction, Defined Class, Intersection
Python rdflib	로딩, SPARQL 질의, 트리플 추가